

Université de Rouen Normandie

Faculté des Sciences et Techniques
Département d'Informatique

Étude algorithmique du problème du voyageur de commerce

Projet d'algorithmique des graphes

Réalisé par

Céline ALLICHE

Fariza HAMITOUCHE

Kenza YOUSFI

Encadré par

Mrs Carla SELMI

Année universitaire 2025–2026

Table des matières

1	Présentation du problème et objectifs	3
2	Modélisation du problème	4
3	Algorithmes d'approximation	5
3.1	Algorithme du Point le Plus Proche (PPP)	5
3.2	Amélioration par décroisement : OptPPP	6
3.3	Approximation par arbre couvrant minimum : OptPrim	7
4	Algorithme exact : HDS	10
5	Structures de données et organisation du projet	12
5.1	Représentation des points du plan	12
5.2	Matrice des distances	12
5.3	Représentation des cycles hamiltoniens	12
5.4	Arbre couvrant minimum	13
5.5	Organisation du projet	13
5.5.1	Module <code>utils.py</code>	13
5.5.2	Module <code>ppp.py</code>	13
5.5.3	Module <code>opt_ppp.py</code>	13
5.5.4	Module <code>prim.py</code>	13
5.5.5	Module <code>hds.py</code>	13
5.5.6	Module <code>main.py</code>	14
6	Résultats expérimentaux et analyse	15
6.1	Protocole expérimental	15
6.2	Résultats numériques	15
6.3	Pourcentages de gain	16
6.4	Comparaison graphique	16
6.5	Analyse et discussion	16
7	Difficultés rencontrées et améliorations possibles	20
7.1	Difficultés rencontrées	20
7.2	Améliorations possibles	20
8	Notice d'utilisation du programme	21
8.1	Pré-requis	21
8.2	Format du fichier des points	21
8.3	Choix du mode d'utilisation	21
8.4	Choix du mode d'exécution	22
8.5	Exécution du programme	22
8.6	Résultats affichés	22

8.7 Arrêt du programme	22
9 Conclusion	23

Chapitre 1

Présentation du problème et objectifs

Le Problème du Voyageur de Commerce (PVC) consiste à déterminer, étant donné un ensemble de points dans le plan, un cycle de longueur minimale passant exactement une fois par chacun de ces points. Une telle solution correspond à un cycle hamiltonien de coût minimal dans un graphe complet pondéré, où le poids des arêtes représente la distance euclidienne entre les points.

Ce problème est connu pour être NP-difficile, ce qui implique qu'il n'existe pas, à l'heure actuelle, d'algorithme polynomial permettant de calculer une solution exacte pour des instances de grande taille. En pratique, le calcul de la solution optimale devient rapidement impossible lorsque le nombre de points augmente.

L'objectif principal de ce projet est donc d'étudier des méthodes approchées permettant d'obtenir des solutions de bonne qualité dans des temps de calcul raisonnables. Plus précisément, ce travail vise à :

- implanter plusieurs algorithmes d'approximation du PVC ;
- comparer leurs performances sur des ensembles de données générés aléatoirement ;
- analyser la qualité des solutions obtenues en les comparant, pour des instances de petite taille, à une solution optimale calculée à l'aide d'un algorithme exact de type *Branch and Bound* (HDS).

Les algorithmes étudiés dans ce projet sont les suivants :

- **PPP (Point le Plus Proche)**, une heuristique gloutonne construisant progressivement un cycle ;
- **OptPPP**, une amélioration locale du cycle obtenu par PPP fondée sur le décroisement des arêtes (méthode 2-opt) ;
- **PVCPrim (ou OptPrim)**, une approximation basée sur la construction d'un arbre couvrant de poids minimum suivi d'un parcours préfixe ;
- **HDS**, un algorithme exact exponentiel, utilisé comme solution de référence pour l'évaluation des heuristiques.

Dans ce projet, l'algorithme exact HDS n'a pas vocation à être utilisé sur des instances de grande taille, en raison de sa complexité exponentielle. Il est employé uniquement pour des instances de petite taille afin de fournir une solution optimale de référence. Cette solution exacte permet d'évaluer objectivement la qualité des algorithmes d'approximation étudiés.

Chapitre 2

Modélisation du problème

Les sommets du graphe sont représentés par des points du plan euclidien. Le graphe considéré est complet et pondéré, le poids d'une arête correspondant à la distance euclidienne entre les deux points associés.

Une solution du PVC est modélisée par un cycle hamiltonien, c'est-à-dire une permutation des sommets formant un cycle fermé.

Cette modélisation est commune à l'ensemble des algorithmes implémentés dans ce projet, aussi bien pour les méthodes d'approximation que pour l'algorithme exact HDS.

Chapitre 3

Algorithmes d'approximation

3.1 Algorithme du Point le Plus Proche (PPP)

L'algorithme du Point le Plus Proche (PPP) construit progressivement un cycle hamiltonien en insérant, à chaque étape, le point non encore visité le plus proche du cycle courant. Il s'agit d'une heuristique gloutonne, simple à implémenter et rapide, mais qui ne garantit pas l'obtention d'une solution optimale.

Pseudo-code :

Procédure PPP(L, Q_0)

Entrée :

- L : ensemble des points $\{P_0, \dots, P_{n-1}\}$;
- Q_0 : point de départ appartenant à L .

Sortie :

- c : cycle hamiltonien ;
- l : longueur du cycle.

Début

```
c ← [Q0]                // cycle initial réduit à un seul sommet
L ← L \ {Q0}            // suppression du point de départ

Tant que L ≠ ∅ faire
    dmin ← +∞            // distance minimale trouvée

    Pour chaque Qi ∈ L faire
        Pour chaque Qj ∈ c faire
            // recherche du point le plus proche du cycle
            Si d(Qi, Qj) < dmin alors
                dmin ← d(Qi, Qj)
                Q ← Qi    // point extérieur le plus proche du cycle
                S ← Qj    // sommet du cycle le plus proche
        Fin Si
    Fin Pour
    Fin Pour

    Insérer Q dans le cycle c immédiatement après S
    L ← L \ {Q}        // Q devient visité
Fin Tant que
```

```

l ← longueur(c)
Retourner (c, l)

```

Fin

Explication globale :

L'algorithme PPP adopte une stratégie gloutonne pour construire le cycle :

- à chaque itération, il sélectionne le point non encore visité qui est le plus proche d'un sommet du cycle courant ;
- ce point est ensuite inséré dans le cycle à l'endroit correspondant au sommet le plus proche ;
- le processus se répète jusqu'à ce que tous les points aient été insérés.

Cette approche permet d'obtenir rapidement une solution admissible du problème du voyageur de commerce. Toutefois, le caractère glouton de l'algorithme peut conduire à des cycles de qualité moyenne, ce qui justifie l'utilisation de méthodes d'amélioration ultérieures, telles que l'algorithme OptPPP.

3.2 Amélioration par décroisement : OptPPP

La procédure OptPPP vise à améliorer un cycle hamiltonien obtenu par l'algorithme PPP en supprimant les croisements d'arêtes lorsque cela permet de réduire la longueur totale du cycle. Cette méthode correspond à une heuristique d'optimisation locale de type *2-opt*, largement utilisée pour le problème du voyageur de commerce.

Pseudo-code :

Procédure OPTPPP(*c*)

Entrée :

- *c* : cycle hamiltonien (liste de sommets indexés de 0 à $n - 1$).

Sortie :

- *c'* : cycle hamiltonien amélioré ;
- *l* : longueur du cycle amélioré.

Début

```

amélioration ← vrai
n ← longueur(c)

// On répète les améliorations tant qu'elles sont possibles
Tant que amélioration faire
    amélioration ← faux

    // Test de tous les couples d'arêtes non adjacentes
    Pour i allant de 0 à n3 faire
        Pour j allant de i+2 à n1 faire

            // Cas interdit : arêtes adjacentes du cycle
            Si (i = 0 et j = n1) alors
                Continuer
            Fin Si

            a ← c[i]
            b ← c[i+1]

```

```

    c1 ← c[j]
    d ← c[(j+1) mod n]

    // Comparaison des coûts avant et après décroisement
    Si d(a, c1) + d(b, d) < d(a, b) + d(c1, d) alors
        Inverser le sous-cycle c[i+1 ... j]
        amélioration ← vrai
    Fin Si
  Fin Pour
Fin Pour
Fin Tant que

l ← longueur(c)
Retourner (c, l)

```

Fin

Explication globale :

La procédure OptPPP repose sur une heuristique locale de type 2-opt :

- elle examine toutes les paires d'arêtes non adjacentes du cycle courant ;
- lorsqu'un décroisement permet de réduire la longueur totale du cycle, celui-ci est effectué en inversant une partie du cycle ;
- le processus est répété jusqu'à ce qu'aucune amélioration supplémentaire ne soit possible.

Cette méthode permet d'améliorer de manière significative les solutions obtenues par PPP.

Bien qu'elle ne garantisse pas l'optimalité, elle est simple à mettre en œuvre et s'avère très efficace en pratique.

3.3 Approximation par arbre couvrant minimum : OptPrim

L'algorithme de Prim permet de construire un arbre couvrant de poids minimum (ACM) d'un graphe pondéré et connexe, à partir d'un sommet racine donné. Dans le cadre du problème du voyageur de commerce, cet arbre est ensuite parcouru afin d'obtenir une tournée hamiltonienne constituant une approximation du cycle optimal.

Pseudo-code de l'algorithme de Prim :

Procédure PRIM(D, r)

Entrée :

- D : matrice des distances entre les sommets ;
- r : sommet racine.

Sortie :

- **parent** : tableau des prédécesseurs définissant l'arbre couvrant.

Début

Pour chaque sommet v faire

clé[v] ← +

parent[v] ← NIL

Fin Pour

clé[r] ← 0

Q ← ensemble de tous les sommets


```

Tant que Q faire
  u ← sommet de Q de clé minimale
  Retirer u de Q

  Pour chaque sommet v tel que v ∈ Q faire
    Si D(u, v) < clé[v] alors
      clé[v] ← D(u, v)
      parent[v] ← u
    Fin Si
  Fin Pour
Fin Tant que

```

Retourner parent

Fin

Explication globale :

L'algorithme de Prim construit progressivement un arbre couvrant minimum :

- à chaque étape, il relie au plus proche sommet de l'arbre un sommet encore extérieur ;
- le résultat est un arbre reliant tous les sommets avec un coût total minimal.

Parcours préfixe de l'arbre couvrant :

À partir de l'arbre couvrant minimum obtenu, on effectue un parcours en profondeur (DFS). L'ordre de visite des sommets fournit alors une tournée hamiltonienne.

Procédure PARCOURSPRÉFIXE(*parent*, *r*)

Entrée :

- *parent* : tableau des prédécesseurs ;
- *r* : racine de l'arbre.

Sortie :

- *ordre* : ordre de visite préfixe des sommets.

Début

ordre ← liste vide

Procédure DFS(*u*)

 Ajouter *u* à *ordre*

 Pour chaque sommet *v* tel que *parent*[*v*] = *u* faire

 DFS(*v*)

 Fin Pour

Fin Procédure

DFS(*r*)

Retourner *ordre*

Fin

Approximation du PVC par OptPrim :

La procédure OptPrim combine les deux étapes précédentes afin d'obtenir une approximation du problème du voyageur de commerce.

Procédure OPTPRIM(*D*, *r*)

Entrée :

- D : matrice des distances ;
- r : sommet de départ.

Sortie :

- c : cycle hamiltonien ;
- l : longueur du cycle.

Début

```
parent ← Prim(D, r)
c ← ParcoursPréfixe(parent, r)
l ← longueur(c)
Retourner (c, l)
```

Fin

Explication globale :

La méthode OptPrim repose sur une propriété fondamentale du PVC métrique :

- le parcours préfixe d'un arbre couvrant minimum fournit une tournée hamiltonienne ;
- grâce à l'inégalité triangulaire, le coût de cette tournée est borné par un facteur constant du coût optimal.

Cette approche fournit en pratique de très bonnes approximations du PVC, tout en restant de complexité polynomiale.

Chapitre 4

Algorithme exact : HDS

L'algorithme HDS repose sur une recherche par *évaluation et séparation progressive* (*Branch and Bound*). Il consiste à parcourir l'arborescence de toutes les permutations possibles des sommets afin de trouver le cycle hamiltonien de coût minimal. Afin de limiter l'explosion combinatoire, les branches dont le coût minimal estimé dépasse la meilleure solution connue sont éliminées prématurément.

Cette méthode permet de calculer une solution exacte du problème du voyageur de commerce, mais sa complexité exponentielle la rend applicable uniquement à des instances de petite taille. Dans ce projet, l'algorithme HDS est utilisé comme solution de référence pour évaluer la qualité des heuristiques d'approximation.

Principe général :

La recherche explore récursivement des chemins partiels correspondant à des permutations incomplètes des sommets. À chaque étape :

- un sommet non encore visité est ajouté au chemin courant ;
- le coût partiel du chemin est mis à jour ;
- une borne inférieure du coût total du cycle est calculée ;
- si cette borne dépasse le coût de la meilleure solution connue, la branche est abandonnée.

Heuristique de la demi-somme :

Pour estimer une borne inférieure du coût d'un cycle partiel, on utilise l'heuristique dite de la *demi-somme*. Pour chaque sommet non encore visité, on considère la somme des deux plus petites distances incidentes à ce sommet. La somme de ces contributions, ajoutée au coût du chemin courant et divisée par deux, fournit une borne inférieure du coût total du cycle.

Cette borne permet d'éliminer efficacement de nombreuses branches de l'arbre de recherche, tout en garantissant que la solution optimale n'est jamais ignorée.

Pseudo-code :

Procédure HDS(D, r)

Entrée :

- D : matrice des distances ;
- r : sommet de départ.

Sortie :

- c^* : cycle hamiltonien optimal ;
- l^* : longueur minimale du cycle.

Début

```
meilleur_cout ← +  
meilleur_cycle ← NIL
```

```

Procédure Explorer(chemin, cout)
    borne ← calcul_borne_demi_somme(chemin, cout)
    Si borne meilleur_cout alors
        Retourner // élagage de la branche
    Fin Si

    Si chemin contient tous les sommets alors
        cout_total ← cout + distance(dernier, r)
        Si cout_total < meilleur_cout alors
            meilleur_cout ← cout_total
            meilleur_cycle ← chemin
        Fin Si
        Retourner
    Fin Si

    Pour chaque sommet v non encore visité faire
        Explorer(chemin + [v], cout + distance(dernier, v))
    Fin Pour
Fin Procédure

Explorer([r], 0)
Retourner (meilleur_cycle, meilleur_cout)

```

Fin

Explication globale :

L'algorithme HDS explore de manière exhaustive l'espace des solutions possibles, tout en réduisant drastiquement le nombre de cas étudiés grâce à l'utilisation de bornes inférieures. Il garantit l'obtention de la solution optimale pour les instances traitées, mais son coût exponentiel limite son utilisation à des problèmes de taille réduite.

Dans ce projet, HDS est utilisé comme référence exacte afin de comparer la qualité des solutions produites par les algorithmes d'approximation PPP, OptPPP et OptPrim.

Cette partie présente l'algorithme exact basé sur une méthode d'évaluation et de séparation progressive (Branch and Bound). L'heuristique de la demi-somme est utilisée pour calculer des bornes inférieures permettant d'élaguer efficacement l'arbre de recherche.

Chapitre 5

Structures de données et organisation du projet

Ce chapitre présente les principales structures de données utilisées dans le projet ainsi que l'organisation générale du code. Les choix effectués visent à assurer à la fois la clarté de l'implémentation, l'efficacité des algorithmes et la facilité d'extension du programme.

5.1 Représentation des points du plan

Les points du plan sont représentés par une liste de tuples de la forme (x, y) , où x et y correspondent aux coordonnées cartésiennes du point. Chaque tuple représente ainsi un sommet du graphe associé au problème du voyageur de commerce.

Ce choix est naturel en Python pour plusieurs raisons :

- les tuples sont des structures immuables, ce qui garantit l'intégrité des coordonnées ;
- l'accès aux coordonnées est direct et lisible ;
- cette représentation est compatible avec les bibliothèques de calcul numérique et d'affichage graphique utilisées dans le projet.

5.2 Matrice des distances

Les distances entre les points sont stockées dans une matrice des distances D , implémentée sous la forme d'une liste de listes. Chaque élément $D[i][j]$ représente la distance euclidienne entre les points i et j .

Cette structure présente plusieurs avantages :

- l'accès aux distances se fait en temps constant ;
- elle évite les recalculs répétés de distances, potentiellement coûteux ;
- elle simplifie l'implémentation des algorithmes du PVC.

5.3 Représentation des cycles hamiltoniens

Un cycle hamiltonien est représenté par une liste d'indices correspondant aux points visités dans l'ordre. Par exemple :

$$c = [i_0, i_1, i_2, \dots, i_{n-1}]$$

Cette représentation :

- facilite les opérations d'insertion et de modification du cycle ;
- permet un calcul simple et efficace de la longueur totale du cycle ;
- est directement exploitable par les algorithmes d'optimisation locale.

5.4 Arbre couvrant minimum

Dans l'algorithme de Prim, l'arbre couvrant de poids minimum est représenté par un tableau des prédécesseurs `parent`, tel que :

$$\text{parent}[v] = u$$

où u est le prédécesseur du sommet v dans l'arbre.

Cette représentation :

- est compacte et peu coûteuse en mémoire ;
- permet une reconstruction simple de l'arbre ;
- est directement exploitable pour effectuer un parcours en profondeur (DFS) afin d'obtenir une tournée hamiltonienne.

5.5 Organisation du projet

Le programme est organisé de manière modulaire, chaque fichier ayant un rôle bien défini. Cette organisation améliore la lisibilité du code, facilite les tests unitaires et permet d'éventuelles extensions futures.

5.5.1 Module `utils.py`

Ce module regroupe les fonctions utilitaires communes à l'ensemble du projet :

- génération aléatoire de points ;
- lecture des points depuis un fichier ;
- calcul de la matrice des distances ;
- affichage graphique des cycles.

5.5.2 Module `ppp.py`

Ce module implémente l'algorithme PPP (Point le Plus Proche). Il construit une solution initiale du problème du voyageur de commerce en insérant itérativement le point non encore visité le plus proche du cycle courant.

5.5.3 Module `opt_ppp.py`

Ce module implémente l'heuristique d'optimisation locale 2-opt. Il permet d'améliorer un cycle existant en supprimant les croisements d'arêtes, conduisant ainsi à des solutions de meilleure qualité à partir du cycle généré par PPP.

5.5.4 Module `prim.py`

Ce module contient :

- l'algorithme de Prim pour le calcul de l'arbre couvrant de poids minimum ;
- la procédure `OptPrim`, qui combine Prim et un parcours préfixe afin d'obtenir une solution approchée du PVC.

5.5.5 Module `hds.py`

Ce module implémente l'algorithme exact HDS (*Held-Demi-Somme*), fondé sur une méthode de recherche par évaluation et séparation progressive (*Branch and Bound*). Il permet de calculer une solution optimale du problème du voyageur de commerce pour des instances de petite taille.

Le module contient :

- l'implémentation de l'heuristique de la demi-somme utilisée pour le calcul de bornes inférieures ;

- la procédure récursive d’exploration des chemins partiels ;
- le mécanisme d’élagage des branches dont la borne dépasse le coût de la meilleure solution connue.

En raison de sa complexité exponentielle, ce module est utilisé uniquement comme référence exacte afin d’évaluer la qualité des solutions produites par les algorithmes d’approximation PPP, OptPPP et OptPrim.

5.5.6 Module `main.py`

Le fichier principal assure :

- l’orchestration des différents algorithmes implémentés ;
- la comparaison des méthodes PPP, OptPPP et OptPrim ;
- la réalisation d’études statistiques portant sur la qualité des solutions obtenues ;
- l’affichage et la visualisation des résultats.

Chapitre 6

Résultats expérimentaux et analyse

Ce chapitre présente les résultats obtenus lors de l'étude expérimentale menée afin de comparer les différentes méthodes d'approximation du problème du voyageur de commerce implémentées dans ce projet. Les algorithmes PPP, OptPPP et OptPrim sont évalués et comparés sur la base de la longueur des cycles obtenus. L'algorithme exact HDS est utilisé comme référence pour des instances de petite taille.

6.1 Protocole expérimental

Les expériences ont été réalisées sur des instances aléatoires du problème du voyageur de commerce. Chaque instance est constituée de $N = 20$ points générés uniformément dans le carré unité $[0, 1] \times [0, 1]$.

Pour chaque instance, les algorithmes PPP, OptPPP et OptPrim ont été exécutés afin de produire une tournée hamiltonienne. La qualité d'une solution est mesurée par la longueur totale du cycle obtenu. L'étude statistique porte sur 100 instances indépendantes.

L'algorithme exact HDS, dont la complexité est exponentielle, est utilisé uniquement sur des instances de petite taille afin de fournir une solution optimale servant de référence pour l'évaluation des heuristiques.

6.2 Résultats numériques

Les longueurs moyennes des cycles obtenus par chaque algorithme, calculées sur l'ensemble des essais, sont présentées dans le tableau 6.1.

Algorithme	Longueur moyenne du cycle
PPP	L_{PPP}
OptPPP	L_{OptPPP}
OptPrim	$L_{OptPrim}$

TABLE 6.1 – Longueurs moyennes des cycles obtenues sur 100 instances

Ces résultats montrent que les algorithmes d'optimisation locale et basés sur les arbres couvrants minimum permettent d'obtenir des solutions de meilleure qualité que l'algorithme PPP de base.

6.3 Pourcentages de gain

Afin de quantifier l'amélioration apportée par les algorithmes OptPPP et OptPrim, les pourcentages de gain sont calculés selon la formule suivante :

$$\text{Gain} = \frac{L_{\text{avant}} - L_{\text{après}}}{L_{\text{avant}}} \times 100$$

Les gains moyens observés sont les suivants :

- le gain de OptPPP par rapport à PPP est de $\frac{L_{\text{PPP}} - L_{\text{OptPPP}}}{L_{\text{PPP}}} \times 100 \%$;
- le gain de OptPrim par rapport à OptPPP est de $\frac{L_{\text{OptPPP}} - L_{\text{OptPrim}}}{L_{\text{OptPPP}}} \times 100 \%$.

Ces résultats confirment l'intérêt des méthodes d'amélioration successives, chaque algorithme permettant de réduire significativement la longueur du cycle obtenu.

6.4 Comparaison graphique

La Figure 6.4 présente une comparaison graphique des longueurs moyennes obtenues par les algorithmes PPP, OptPPP et OptPrim sur l'ensemble des instances testées.

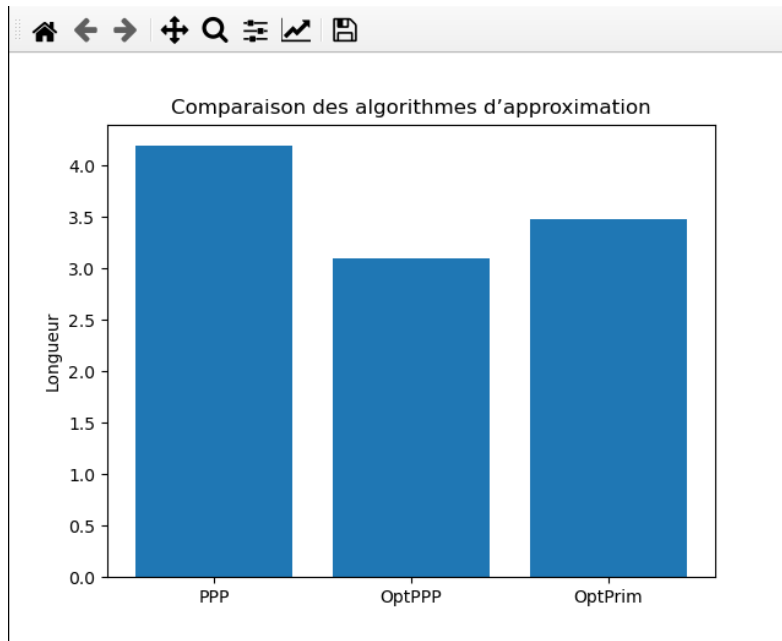


FIGURE 6.1 – Comparaison des algorithmes d'approximation du PVC

6.5 Analyse et discussion

Cette section propose une analyse qualitative des solutions obtenues par les différents algorithmes d'approximation du problème du voyageur de commerce. Les figures suivantes illustrent des cycles représentatifs produits par chaque méthode et permettent de mieux comprendre les différences observées dans les résultats numériques.

Cycle obtenu par l'algorithme PPP

Le cycle obtenu par l'algorithme PPP présente plusieurs croisements d'arêtes, ce qui explique sa longueur relativement élevée. Ce comportement est caractéristique des heuristiques gloutonnes,

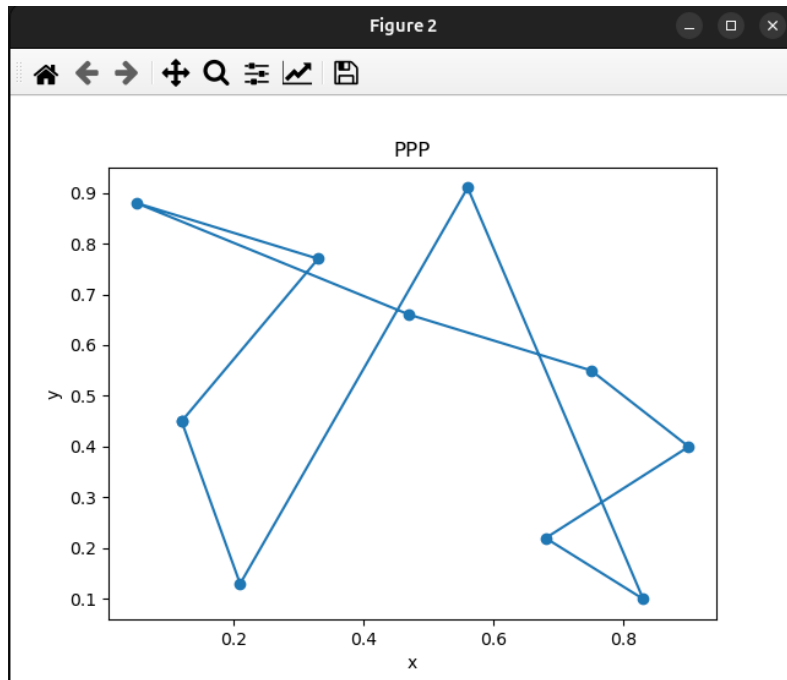


FIGURE 6.2 – Exemple de cycle obtenu par l’algorithme PPP

qui prennent des décisions locales sans corriger les choix sous-optimaux effectués aux étapes précédentes.

Cycle obtenu par l’algorithme OptPPP

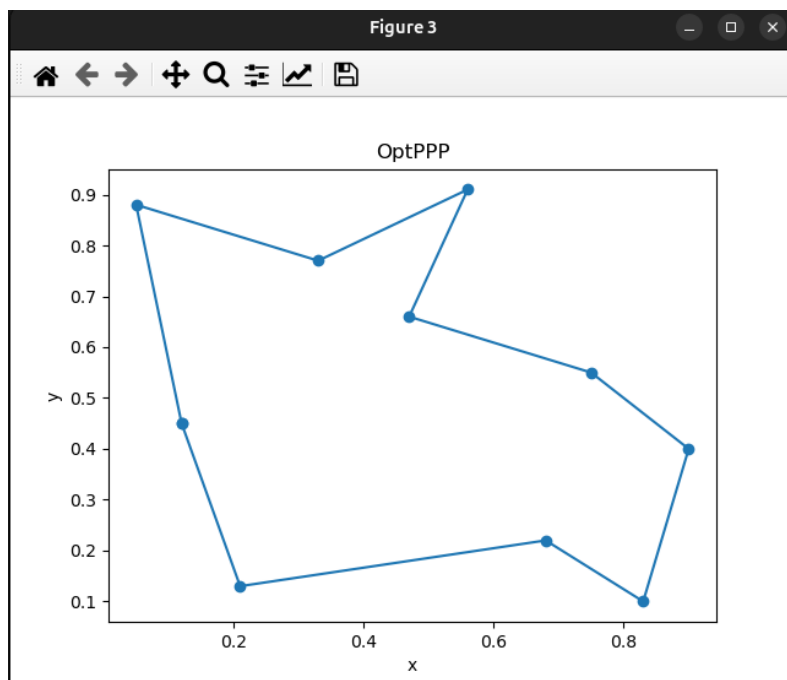


FIGURE 6.3 – Exemple de cycle obtenu après amélioration par OptPPP

Le cycle obtenu après application de l’algorithme OptPPP ne présente plus de croisements d’arêtes. L’heuristique locale de type 2-opt permet de corriger les choix sous-optimaux effectués

par l'algorithme PPP et d'améliorer significativement la longueur totale du cycle. Cette méthode offre un excellent compromis entre qualité de solution et coût de calcul.

Cycle obtenu par l'algorithme OptPrim

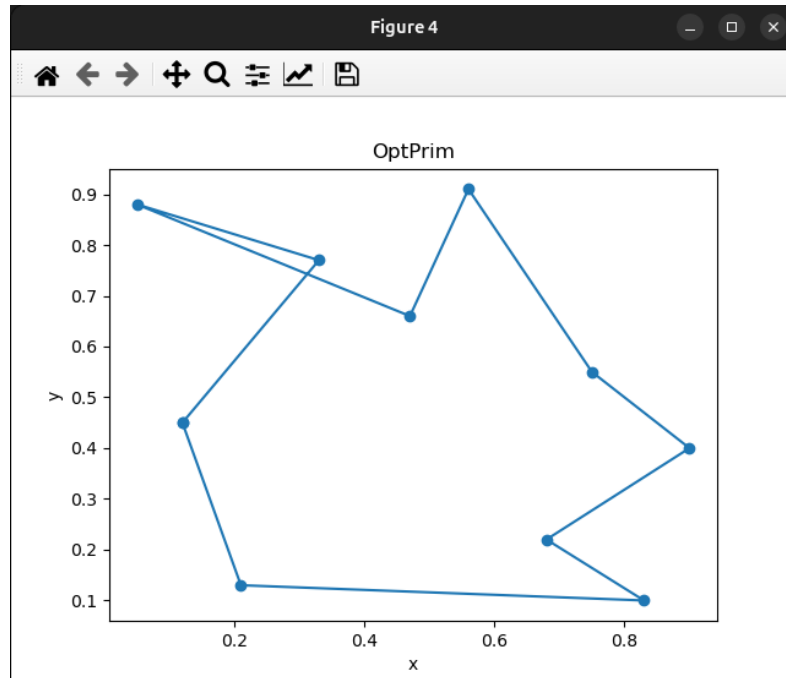


FIGURE 6.4 – Exemple de cycle obtenu par l'algorithme OptPrim

Le cycle produit par l'algorithme OptPrim est issu du parcours préfixe d'un arbre couvrant de poids minimum. Cette approche permet de respecter la proximité géométrique entre les points et conduit généralement à des solutions de bonne qualité. Bien que cette méthode ne garantisse pas l'optimalité, elle fournit des résultats compétitifs tout en conservant une complexité polynomiale.

Enfin, la comparaison avec l'algorithme exact HDS sur des instances de petite taille montre que les heuristiques proposées produisent des solutions proches de l'optimum, ce qui confirme leur efficacité en pratique.

Cycle optimal obtenu par l'algorithme HDS

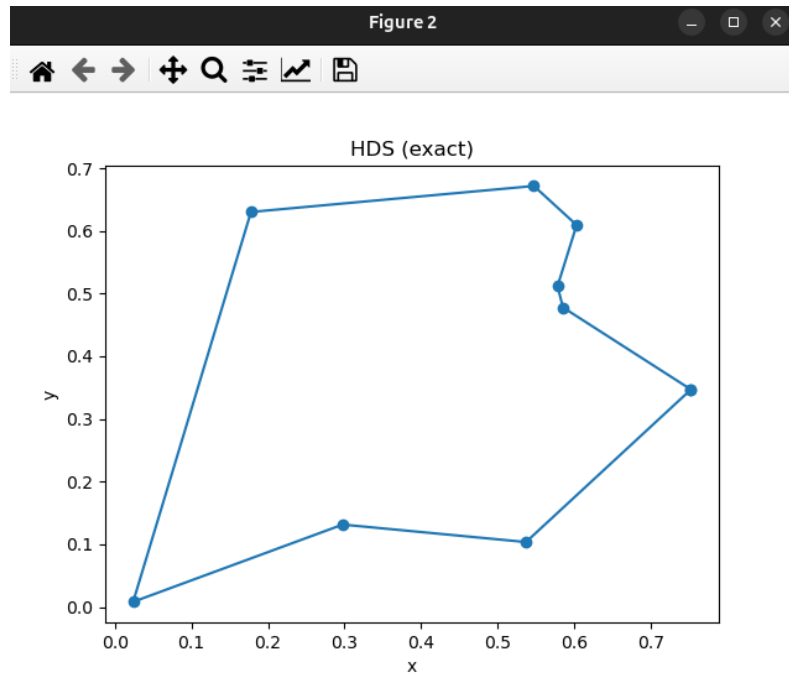


FIGURE 6.5 – Cycle hamiltonien optimal obtenu par l'algorithme exact HDS

Le cycle obtenu par l'algorithme HDS correspond à la solution optimale du problème du voyageur de commerce pour cette instance. Il ne présente aucun croisement d'arêtes et minimise la longueur totale de la tournée.

La comparaison visuelle entre ce cycle optimal et ceux produits par les algorithmes d'approximation montre que ces derniers fournissent des solutions très proches de l'optimum, en particulier pour OptPPP et OptPrim, tout en présentant des temps de calcul bien inférieurs.

Chapitre 7

Difficultés rencontrées et améliorations possibles

7.1 Difficultés rencontrées

La première difficulté a concerné la mise en œuvre correcte des algorithmes théoriques étudiés en cours. En particulier, la traduction des pseudo-codes en Python a nécessité une attention particulière afin de respecter fidèlement les principes algorithmiques tout en tenant compte des spécificités du langage.

Une seconde difficulté a porté sur la gestion des structures de données. Le choix de représentations simples et efficaces pour les points, les cycles et la matrice des distances a été essentiel pour garantir la lisibilité du code et la performance des algorithmes. Des erreurs initiales de manipulation des indices ont notamment nécessité plusieurs phases de correction et de validation.

La mise en place des études expérimentales a également soulevé des difficultés, en particulier dans la gestion des entrées depuis des fichiers externes. Il a fallu rendre la lecture des données suffisamment robuste pour prendre en compte les variations de format et éviter des erreurs d'exécution.

Enfin, la génération et l'organisation des figures dans le rapport ont demandé une attention particulière. La gestion du placement automatique des figures par $\text{\LaTeX}$ a nécessité plusieurs ajustements afin d'obtenir une mise en page cohérente et lisible.

7.2 Améliorations possibles

Plusieurs améliorations pourraient être envisagées pour prolonger ce travail. En premier lieu, il serait possible d'intégrer des heuristiques d'optimisation plus avancées, telles que les méthodes 3-opt ou les algorithmes métaheuristiques (recuit simulé, algorithmes génétiques), afin d'obtenir des solutions de meilleure qualité pour des instances de grande taille.

Une autre piste d'amélioration consisterait à étudier plus finement les performances en temps de calcul des différents algorithmes, en mesurant explicitement les temps d'exécution et en analysant leur évolution en fonction du nombre de points.

Il serait également envisageable d'améliorer la visualisation des résultats, par exemple en produisant des graphiques interactifs ou en comparant automatiquement les cycles obtenus avec la solution optimale lorsque celle-ci est disponible.

Enfin, l'algorithme exact HDS pourrait être optimisé ou parallélisé afin de traiter des instances légèrement plus grandes, ou être utilisé pour calibrer et évaluer plus précisément les performances des heuristiques d'approximation.

Chapitre 8

Notice d'utilisation du programme

Cette section décrit les prérequis nécessaires, le format des données d'entrée, les modes d'utilisation ainsi que le déroulement de l'exécution du programme développé dans le cadre de ce projet.

8.1 Pré-requis

Pour exécuter correctement le programme, il est nécessaire de disposer des éléments suivants :

- Un environnement **Python 3** ;
- L'IDE **Pyzo** (ou tout autre IDE compatible avec Python) ;
- Les bibliothèques Python suivantes :
 - `numpy`
 - `matplotlib`

Ces bibliothèques sont généralement installées par défaut. Si ce n'est pas le cas, elles peuvent être installées à l'aide de la commande suivante :

```
pip install numpy matplotlib
```

8.2 Format du fichier des points

Le programme peut fonctionner à partir d'un fichier texte contenant les coordonnées des points. Chaque ligne du fichier `points.txt` doit contenir un point au format suivant :

(x, y)

Exemple :

```
(0.12, 0.45)
(0.78, 0.23)
(0.34, 0.91)
(0.56, 0.67)
```

Les coordonnées doivent être comprises entre 0 et 1.

8.3 Choix du mode d'utilisation

Le mode d'entrée des points est défini dans le fichier `main.py` à l'aide des paramètres suivants :

```
MODE_ENTREE = "fichier"      # "aleatoire" ou "fichier"
FICHIER_POINTS = "points.txt"
```

- `MODE = "aleatoire"` : les points sont générés aléatoirement dans l'intervalle $[0, 1]$;
- `MODE = "fichier"` : les points sont lus depuis le fichier `points.txt`.

8.4 Choix du mode d'exécution

Le mode d'exécution est défini dans le fichier `main.py` à l'aide des paramètres suivants :

```
MODE_EXECUTION = "approx"    # "approx" ou "hds"  
FICHIER_POINTS = "points_hds.txt"
```

Le nombre de points et le nombre d'essais statistiques peuvent également être modifiés :

```
N = 2012  
NB_ESSAIS = 100
```

8.5 Exécution du programme

Pour exécuter le programme :

1. Ouvrir Pyzo ;
2. Charger le fichier `main.py` ;
3. Vérifier que le dossier de travail correspond au dossier du projet ;
4. Lancer l'exécution du script.

Le programme effectue alors automatiquement :

- l'exécution des algorithmes PPP, OptPPP et OptPrim ;
- une étude statistique sur 100 essais ;
- le calcul des longueurs moyennes et des pourcentages de gain ;
- l'affichage d'un graphique comparatif ;
- la visualisation des cycles obtenus pour un cas concret.

8.6 Résultats affichés

Le programme affiche dans la console :

- les longueurs moyennes obtenues par chaque algorithme ;
- les pourcentages de gain entre les méthodes.

Des fenêtres graphiques s'ouvrent également pour :

- comparer visuellement les longueurs moyennes (diagramme en barres) ;
- afficher les cycles hamiltoniens obtenus par PPP, OptPPP et OptPrim.

8.7 Arrêt du programme

Le programme se termine automatiquement après l'affichage des résultats et des figures. Il suffit de fermer les fenêtres graphiques pour revenir à l'environnement Pyzo.

Chapitre 9

Conclusion

Ce projet a permis d'étudier le problème du voyageur de commerce à travers l'implémentation et la comparaison de plusieurs algorithmes d'approximation. Les méthodes PPP, OptPPP et OptPrim ont été implantées conformément aux notions vues en cours et évaluées sur différentes instances de points.

Les résultats expérimentaux montrent que l'algorithme PPP permet d'obtenir rapidement une solution admissible, mais que sa qualité reste limitée. L'heuristique OptPPP améliore efficacement ces solutions en supprimant les croisements d'arêtes, tandis que l'algorithme OptPrim fournit en moyenne des solutions de bonne qualité tout en conservant une complexité polynomiale.

L'utilisation de l'algorithme exact HDS sur des instances de petite taille a permis de comparer les solutions approchées à l'optimum et de confirmer la pertinence des heuristiques étudiées. Ce projet a ainsi permis de mettre en pratique les concepts d'algorithmique des graphes et d'en comprendre les enjeux en situation concrète.